UNIVERSITY SCHOOL OF AUTOMATION AND ROBOTICS
GURU GOBIND SINGH INDRAPRASTHA UNIVERSITY
EAST DELHI CAMPUS, DELHI

A SYNOPSIS

ON

"MULTIMODAL RETRIEVAL-AUGMENTED GENERATION (RAG) BASED AI SYSTEM"

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted By:

Varnit	Enrollment No.: 12919051623
Dev Sharma	Enrollment No.: 10219051623

Under the Guidance of:

Dr. Manisha Jangid	Faculty Mentor

Academic Session 2026–27

Contents

Abstract..... 3
1 Introduction..... 4
2 Literature Review..... 5
3 Problem Statement..... 6
4 Objectives..... 7
5 Scope of the Project..... 8
6 Proposed System..... 9
6.1 Document Ingestion Module..... 9
6.2 Multimodal Embedding Module..... 9
6.3 Vector Storage and Retrieval Module..... 10
6.4 Query Processing Module..... 10
6.5 Context Assembly and Generation Module..... 10
6.6 Response and Source Attribution Module..... 10
6.7 User Interface Module..... 10
7 Proposed Methodology..... 11
7.1 Phase 1: Requirement Analysis..... 11
7.2 Phase 2: Literature and Tool Study..... 11
7.3 Phase 3: System Design..... 11
7.4 Phase 4: Backend Development..... 11
7.5 Phase 5: Frontend Development..... 11
7.6 Phase 6: Testing and Evaluation..... 11
7.7 Phase 7: Documentation..... 11
8 System Architecture..... 12
9 Functional Requirements..... 13
10 Non-Functional Requirements..... 14
11 Technology Stack..... 15
12 Hardware and Software Requirements..... 16
13 Expected Outcomes..... 17
14 Future Scope..... 18
15 Work Plan..... 19
16 Conclusion..... 20
References..... 21

Abstract

Large Language Models (LLMs) have transformed the way information is generated and consumed, but they remain limited by fixed training knowledge, a tendency to produce factually incorrect or fabricated content (“hallucination”), and an inability to reason directly over a user’s own documents. Retrieval-Augmented Generation (RAG) addresses these limitations by combining a retrieval mechanism with a generative model, grounding the model’s output in relevant, externally stored content before an answer is generated.

Most real-world documents, however, are not purely textual — they contain images, diagrams, and tables alongside text. This project proposes a Multimodal RAG based AI System that extends the standard RAG pipeline to ingest, embed, store, and retrieve both text and image content from documents, and uses this combined context to generate accurate, source-grounded answers to natural-language queries.

The project is built entirely on tools that are widely adopted in the current Generative AI industry and free to use: LangChain for orchestrating the retrieval-generation pipeline, open-source Hugging Face models (Sentence-Transformers and CLIP) for text and image embeddings, Pinecone as the vector database, and an open-source LLM (such as Meta Llama 3.1 or Mistral) served through the Hugging Face Inference API for answer generation. The goal is to build a working, well-understood multimodal RAG system using popular, well-documented, industry-standard, and cost-free components rather than paid proprietary APIs, so that the project closely mirrors how RAG systems are actually built in practice today without incurring API usage costs.

The system will allow a user to upload documents, ask natural-language questions about their content — including questions that depend on information present in embedded images or tables — and receive grounded answers along with references to the exact source used, making the process transparent and verifiable.

1 Introduction

Generative Artificial Intelligence, particularly open Large Language Models such as those hosted on Hugging Face, has rapidly become central to how individuals and organizations search for information, summarize content, and automate knowledge work. These models are pre-trained on massive corpora and can produce fluent, human-like responses to a wide variety of prompts. However, an LLM used on its own has two fundamental limitations: its knowledge is frozen at the time of training, and it has no direct access to private, recent, or domain-specific documents that a user may want answers about.

Retrieval-Augmented Generation (RAG) was introduced to address exactly this gap. Rather than relying purely on parameters learned during pre-training, a RAG system first retrieves relevant pieces of information from an external knowledge source — typically stored as vector embeddings in a vector database — and then supplies this retrieved context to the LLM as part of its prompt. The model's response is thereby "grounded" in real, verifiable content, which significantly reduces hallucination and allows the system to answer questions about material it was never trained on.

Most introductory RAG systems operate only on plain text. In practice, however, a large proportion of useful documents — research papers, textbooks, manuals, reports — combine text with diagrams, charts, screenshots, and tables. A text-only RAG pipeline simply ignores this visual information, even when the answer to a user's question is contained in a figure or a table rather than in a paragraph. Multimodal RAG extends the retrieval-and-generation pipeline to embed and retrieve non-text content as well, so that visual and tabular information can also contribute to the final, generated answer.

This project, Multimodal RAG based AI System, focuses on designing and implementing such a system using tools that are widely adopted in the current Generative AI industry and available free of cost — LangChain for pipeline orchestration, Hugging Face Sentence-Transformers and CLIP for embeddings, Pinecone as the vector database, and an open-source LLM (e.g., Llama 3.1 or Mistral) served via the Hugging Face Inference API for generation — so that a beginner learns the same, industry-standard building blocks used in real-world, production multimodal RAG systems, without depending on paid proprietary APIs.

The project also provides an opportunity to study foundational Generative AI and Information Retrieval concepts, including vector embeddings, semantic similarity search, prompt construction, context-window management, cross-modal retrieval, and the practical trade-offs between model size, accuracy, and computational cost.

2 Literature Review

Retrieval-Augmented Generation was formally introduced as a way to combine parametric knowledge stored in a pre-trained language model with non-parametric knowledge retrieved from an external corpus at inference time, improving factual accuracy on knowledge-intensive tasks without requiring the model itself to be retrained. This retriever-generator pattern — first explored in open-domain question answering — remains the foundation of nearly all modern RAG systems.

A key enabler of effective retrieval is Dense Passage Retrieval (DPR), which uses dual encoders to map queries and documents into a shared vector space so that semantic relevance can be measured using vector similarity rather than exact keyword overlap. This idea generalizes directly to multimodal settings: contrastive vision-language models such as CLIP (Contrastive Language-Image Pre-training) and BLIP learn a joint embedding space for images and text, so that a text query can retrieve semantically related images and vice-versa. Sentence-embedding models such as Sentence-BERT play the equivalent role on the text side, producing compact, semantically meaningful vectors that preserve similarity over full sentences and paragraphs rather than individual tokens.

Once embeddings are produced, they are typically stored in a vector database that supports Approximate Nearest Neighbor (ANN) search, allowing the system to retrieve the top-k most relevant chunks for a query efficiently even as the corpus grows. Lightweight, easy-to-run options such as ChromaDB and FAISS are widely used for local or small-scale deployments, in contrast to managed, large-scale vector stores used in enterprise systems.

Contemporary RAG architectures extend the basic retriever-generator pipeline with techniques such as iterative retrieval and re-ranking, where an initial set of retrieved chunks is refined or re-scored before being passed to the generator, and hybrid indexing, which combines sparse keyword-based search (e.g., BM25) with dense vector search to capture both exact terminology and semantic meaning. Multimodal RAG research further shows that once text and image embeddings share a common vector space, a single retrieval step can surface a mixture of text and image chunks that are jointly relevant to a query, which the generator then synthesizes into one coherent answer.

Orchestration frameworks such as LangChain have become the de-facto standard for building RAG applications, providing ready-made abstractions for document loading, chunking, embedding, vector-store integration, and prompt assembly, which significantly reduces the amount of boilerplate code required to connect these components together. On the vector-storage side, managed, cloud-native databases such as Pinecone have become widely used in production RAG systems because they remove the operational burden of running and scaling a vector index, while still supporting fast Approximate Nearest Neighbor search and metadata filtering. This combination of a popular orchestration framework, a managed vector database, and a leading LLM API directly motivates the design choices of the proposed project, which favours widely adopted, well-documented, industry-standard tools over custom-built alternatives.

Finally, the literature consistently identifies the LLM's finite context window as a practical constraint: only a limited number of tokens can be supplied as retrieved context, which makes careful chunking, top-k selection, and concise context assembly essential parts of any RAG pipeline — multimodal or otherwise.

3 Problem Statement

Standalone Large Language Models are prone to hallucination and cannot answer questions about documents outside their training data, which makes them unreliable for tasks that require factual grounding in a specific document set. Existing open-source RAG tutorials and starter projects almost always restrict themselves to plain text, discarding images, diagrams, and tables that are frequently essential to understanding a document — for example, a data trend shown only in a chart, or a specification listed only in a table.

At the same time, most published multimodal RAG systems that do support images are built around large, resource-intensive embedding and generation models and are deployed on substantial cloud infrastructure. This makes them difficult for a student or beginner to set up, run, and study without significant computational resources or cloud spending.

There is, therefore, a need for a system that can:

- Ingest documents containing a mix of text, images, and tables.
- Convert both text and image content into a shared, comparable vector representation.
- Store and retrieve the most relevant multimodal content for a given natural-language query.
- Use a widely used, industry-standard LLM API to generate an answer that is grounded in the retrieved multimodal context.
- Present the generated answer along with a reference to the exact source (document, page, and content type) used to produce it.
- Run on modest, student-accessible hardware without requiring expensive cloud infrastructure.

The proposed project addresses this gap by designing a multimodal RAG pipeline built entirely from widely used, industry-standard, and free-to-use components — LangChain, Hugging Face Sentence-Transformers and CLIP embeddings, Pinecone, and an open-source LLM served via the Hugging Face Inference API — that are well documented and closely reflect current practice in the Generative AI industry.

4 Objectives

The primary objective of the project is to design and implement a Multimodal Retrieval-Augmented Generation system, built using widely adopted industry-standard tools, that can answer natural-language questions grounded in the text and image content of a given set of documents.

The major objectives are:

1. To study the fundamentals of Retrieval-Augmented Generation, vector embeddings, and semantic similarity search.

2. To study multimodal embedding techniques that allow text and images to be represented in a shared vector space.

3. To develop a document ingestion pipeline capable of extracting text, images, and tables from PDF documents.

4. To generate vector embeddings for extracted text and image chunks using widely used, open-source Hugging Face embedding models (Sentence-Transformers and CLIP).

5. To design and implement a retrieval mechanism that returns the most relevant multimodal chunks for a given user query.

6. To integrate a leading, widely used LLM API that generates a grounded, context-aware answer from the retrieved chunks.

7. To attach source references (document, page, and content type) to every generated answer for transparency and verifiability.

8. To build a simple, beginner-friendly user interface for uploading documents and interacting with the system.

9. To evaluate the system’s retrieval accuracy, answer quality, and response time on a set of sample documents.

5 Scope of the Project

The scope of the proposed project covers the design and implementation of a multimodal RAG pipeline, built on widely used industry-standard tools, operating on user-supplied PDF documents in a controlled, single-user (or small-group) demonstration environment.

The system will primarily focus on:

- Extraction of text, images, and basic tabular content from PDF documents.
- Chunking of extracted text into embedding-friendly segments.
- Generation of text embeddings using a widely used, open-source Hugging Face embedding model (Sentence-Transformers).
- Generation of image embeddings using CLIP, a widely used vision-language embedding model.
- Storage of embeddings and associated metadata in Pinecone, a leading managed vector database.
- Semantic similarity search (retrieval) across text and image embeddings for a given query.
- Assembly of retrieved multimodal context, using LangChain, into a prompt for a leading generative LLM API.
- Generation of a grounded answer along with source attribution.
- A simple web-based interface for document upload and question-answering.

The project will not attempt to support audio or video modalities, will not implement large-scale distributed infrastructure, and will not train or fine-tune a language model from scratch. The system is intended as an educational and demonstrative implementation rather than a production-scale enterprise deployment; nonetheless, its modular design keeps it extensible toward those directions in the future.

6 Proposed System

The proposed system consists of seven cooperating modules that together implement the multimodal RAG pipeline, from document ingestion through to a grounded, source-attributed answer. Figure 1 illustrates the end-to-end ingestion and query workflow, drawn with straight, right-angled connector lines for clarity.

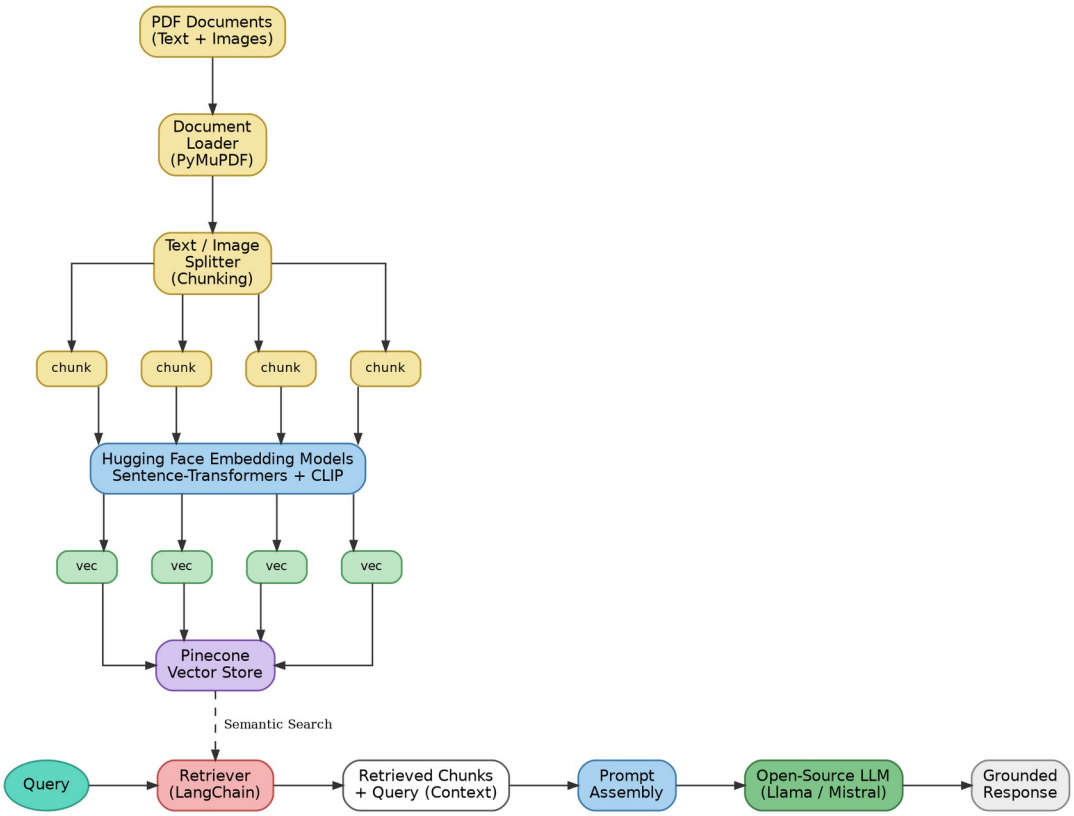


Figure 1: Multimodal RAG — Ingestion and Query Workflow

6.1 Document Ingestion Module

This module accepts one or more PDF documents supplied by the user. It uses a PDF-parsing library to walk through each page, extracting raw text, embedded images, and basic tabular structures, and tags each extracted item with metadata such as source filename, page number, and content type.

6.2 Multimodal Embedding Module

Extracted text is split into manageable chunks and passed through a Hugging Face Sentence-Transformers model to produce dense text vectors. Extracted images are passed through CLIP (accessed via the Hugging Face Transformers library) to produce image vectors in the same shared embedding space, enabling cross-modal similarity comparisons between a text query and image content. Both models are open-source and free to run, either locally or through the Hugging Face Inference API.

6.3 Vector Storage and Retrieval Module

All embeddings, together with their metadata, are persisted in Pinecone, a managed cloud vector database. When a query arrives, the module performs an Approximate Nearest Neighbor search to retrieve the top-k most relevant text and image chunks.

6.4 Query Processing Module

This module accepts the user's natural-language question, embeds it using the same text embedding model used for ingestion, and passes the resulting query vector to the retrieval module.

6.5 Context Assembly and Generation Module

Retrieved text and image chunks are grouped and assembled into a structured prompt along with the user's original question, using LangChain's retrieval-chain abstractions. This prompt is passed to an open-source LLM (such as Meta Llama 3.1 or Mistral) served through the free Hugging Face Inference API, which synthesizes a coherent, context-grounded answer.

6.6 Response and Source Attribution Module

The generated answer is returned together with references to the specific document, page number, and content type (text, image, or table) that contributed to it, so that the user can verify the answer against the original source.

6.7 User Interface Module

A Streamlit-based web interface — one of the most widely used frameworks for building AI application front-ends — allows the user to upload documents, ask questions in natural language, and view the generated answer alongside its cited sources in a chat-like format.

7 Proposed Methodology

The project will follow a modular, iterative development methodology, broken into the following phases.

7.1 Phase 1: Requirement Analysis

Studying core Generative AI and Information Retrieval concepts, defining functional and non-functional requirements, and identifying the set of sample documents to be used for development and testing.

7.2 Phase 2: Literature and Tool Study

Studying existing RAG frameworks and widely adopted industry tools, including LangChain, Hugging Face Sentence-Transformers and CLIP embedding models, managed vector databases such as Pinecone, and open-source LLMs (Llama 3.1, Mistral) available through the Hugging Face Inference API.

7.3 Phase 3: System Design

Designing the modular system architecture covering document ingestion, embedding, vector storage, retrieval, generation, and the user interface.

7.4 Phase 4: Backend Development

Implementing the ingestion pipeline, embedding generation, vector database integration, retrieval logic, prompt assembly, and LLM integration in Python.

7.5 Phase 5: Frontend Development

Building a Streamlit-based web interface for document upload and question-answering, designed for ease of use by a beginner while reflecting current industry-standard front-end tooling for AI applications.

7.6 Phase 6: Testing and Evaluation

Testing the system on a curated set of multimodal documents, evaluating retrieval relevance, answer accuracy, source-attribution correctness, and response time.

7.7 Phase 7: Documentation

Documenting the architecture, implementation details, experiments, results, limitations, and possible future improvements.

8 System Architecture

The proposed architecture consists of the following major components:

1. User Interface — for document upload and query submission.
2. Backend API — coordinates the ingestion and query-answering workflow.
3. Document Parser — extracts text, images, and tables from uploaded PDFs.
4. Multimodal Embedding Engine — generates text and image vector embeddings.
5. Vector Database — stores embeddings and metadata; performs similarity search.
6. Generation Engine (LLM) — produces the final grounded answer.
7. Response Formatter — attaches source references to the generated answer.

The general flow of the system is as follows:

User → Uploads Document → Document Parser → Multimodal Embedding Engine → Vector Database (storage). User → Submits Query → Query Embedding → Vector Database (retrieval) → Context Assembly → Generation Engine (LLM) → Response Formatter → User Interface.

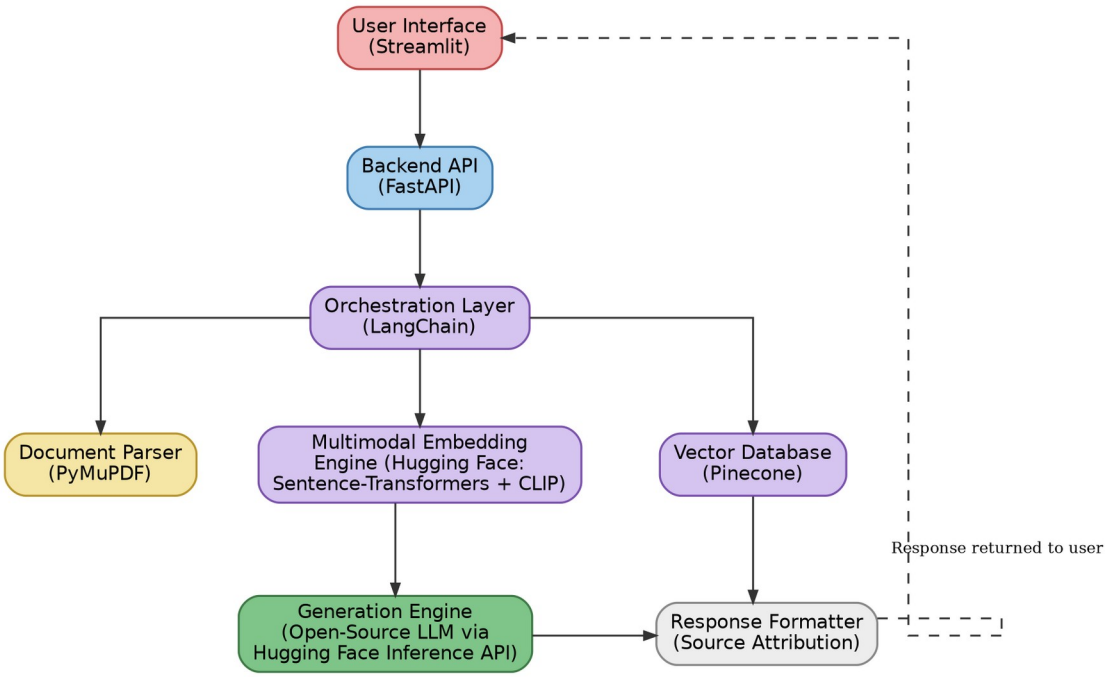


Figure 2: System Architecture — Component Diagram (straight, right-angled connectors)

9 Functional Requirements

The proposed system shall provide the following functionality:

1. The user shall be able to upload one or more PDF documents.
2. The system shall extract text, images, and tables from the uploaded documents.
3. The system shall generate vector embeddings for extracted text and image content.
4. The system shall store embeddings and metadata in a vector database.
5. The user shall be able to submit natural-language questions about the uploaded documents.
6. The system shall retrieve the most relevant text and image chunks for a given question.
7. The system shall generate a context-grounded answer using a language model.
8. The system shall display the source document, page number, and content type used to generate each answer.
9. The system shall present the interaction through a simple, chat-style interface.

10 Non-Functional Requirements

10.1 Security

API keys for the embedding, vector database, and LLM providers should be stored securely (e.g., as environment variables) and never hard-coded, and the system should avoid sending or storing sensitive document content beyond what is required for retrieval and generation.

10.2 Performance

Embedding generation and retrieval should complete within a reasonable time for small and medium-sized document sets, leveraging the performance of managed cloud services such as Pinecone and the chosen LLM API.

10.3 Usability

The interface should be simple enough for a beginner to upload documents and ask questions without technical setup knowledge.

10.4 Reliability

The system should handle unsupported file content or partially extractable pages gracefully, without failing the entire ingestion process.

10.5 Maintainability

The modular architecture should allow individual components — such as the embedding model or the language model — to be replaced or upgraded independently.

10.6 Scalability

The architecture should allow additional modalities, larger document sets, or alternative open-source models (e.g., switching between Llama and Mistral, or between Pinecone and another vector database) to be introduced in future versions without a fundamental redesign.

11 Technology Stack

Component	Technology
Programming Language	Python
Backend Framework	FastAPI
Orchestration Framework	LangChain (industry-standard RAG orchestration framework)
Frontend / UI	Streamlit (widely used for AI application interfaces)
Document Parsing	PyMuPDF (fitz)
Text Embedding Model	Hugging Face Sentence-Transformers — all-MiniLM-L6-v2 (free, open-source, widely used)
Image Embedding Model	CLIP — via Hugging Face Transformers (free, open-source vision-language embedding model)
Vector Database	Pinecone (leading managed, cloud-native vector database used in production RAG systems)
Generation Model (LLM)	Open-source LLM — Meta Llama 3.1 / Mistral, served via the Hugging Face Inference API (free)
Data Processing	Pandas, NumPy
Version Control	Git and GitHub
Development Environment	Visual Studio Code / Google Colab

12 Hardware and Software Requirements

12.1 Hardware Requirements

- Processor: Intel Core i5 / AMD Ryzen 5 or equivalent.
- RAM: Minimum 8 GB.
- GPU: Not required for the free-tier setup — embedding and answer generation are handled through the free Hugging Face Inference API, and Pinecone hosts the vector index in the cloud, so the system runs comfortably on ordinary laptop hardware; a GPU is only needed if the embedding or LLM models are run locally instead.
- Storage: Minimum 5 GB available space.
- A stable internet connection is required, since embedding generation, vector storage, and answer generation all rely on cloud APIs.

12.2 Software Requirements

- Operating System: Windows / Linux / macOS.
- Python 3.10 or above.
- Git and GitHub.
- Visual Studio Code.
- LangChain, Pinecone client, and Hugging Face (transformers / huggingface_hub) libraries.
- API keys for the chosen embedding, vector database, and LLM providers.
- Modern web browser.

13 Expected Outcomes

The expected outcome of the project is a functional multimodal RAG system, built on widely used industry-standard tools, capable of answering natural-language questions grounded in the text and image content of a given set of documents.

The system is expected to:

- Correctly extract text, images, and tables from sample PDF documents.
- Generate meaningful vector embeddings for both text and image content.
- Retrieve relevant multimodal context for a wide range of natural-language queries.
- Generate accurate, context-grounded answers with correct source attribution.
- Run reliably on typical student-grade hardware.
- Demonstrate, in a beginner-accessible way, the core principles used in production-grade multimodal RAG systems.

The project will also provide practical, hands-on understanding of vector embeddings, semantic search, prompt engineering, multimodal AI, and the architecture of modern Generative AI applications.

14 Future Scope

The project can be extended in several directions. Future versions may incorporate hybrid retrieval combining dense and keyword-based (sparse) search, re-ranking of retrieved chunks for higher precision, and support for additional modalities such as audio and video.

The system could also be scaled up by moving to higher-tier Pinecone plans and more capable LLM configurations for multi-user, production-grade deployment. Further enhancements could include fine-tuning or system-prompt specialization for domain-specific data, adding conversational memory across multiple queries using LangChain's memory modules, and integrating the system into existing knowledge-management or e-learning platforms.

15 Work Plan

The project will be completed over a 12-week schedule spanning nine phases, from requirement analysis through to final documentation. The Gantt chart below shows the planned duration and sequencing of each phase by project week.

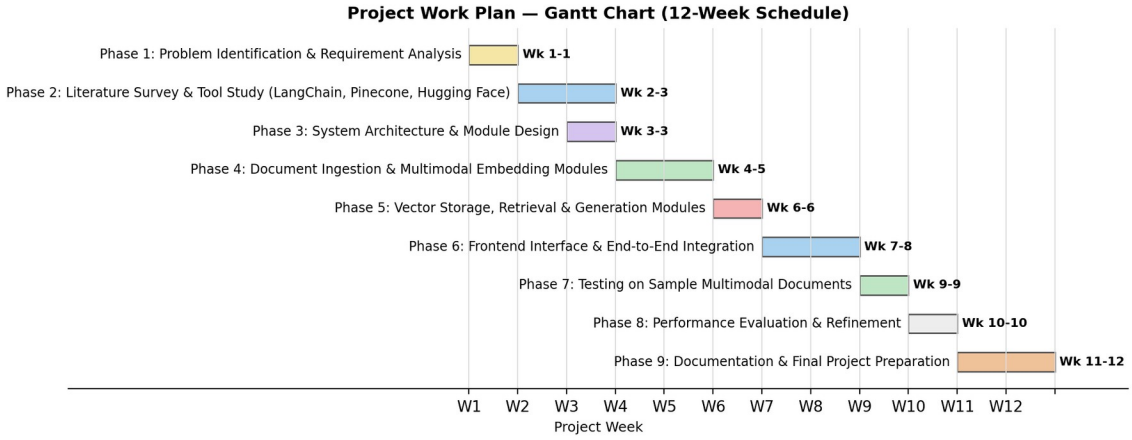


Figure 3: Project Work Plan — Gantt Chart (12-Week Schedule)

16 Conclusion

Generative AI systems are most useful when their outputs can be trusted and traced back to real, verifiable sources, and Retrieval-Augmented Generation provides a practical way to achieve this by grounding language-model responses in externally retrieved content. Extending this idea to the multimodal case — incorporating images and tables alongside text — makes the resulting system far more capable of handling real-world documents, where important information is often not confined to plain text.

The proposed Multimodal RAG based AI System aims to implement this pipeline using LangChain, Hugging Face Sentence-Transformers and CLIP embeddings, Pinecone, and an open-source LLM served via the Hugging Face Inference API — the same widely used, industry-standard, and free-to-use tools used in real-world RAG development — making it both a practical, cost-free learning project and directly relevant to current industry practice. The project combines document ingestion, multimodal embedding, vector-based retrieval, and grounded generation into a single, understandable, and reproducible system.

Beyond the working system itself, the project is expected to provide a strong practical foundation in Generative AI, Information Retrieval, and multimodal machine learning — concepts that underpin many of today's most impactful AI applications — while leaving clear directions for future scaling and enhancement.

References

1. Lewis, P., Perez, E., et al. (2021). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” arXiv:2005.11401.

2. Karpukhin, V., Oguz, B., et al. (2020). “Dense Passage Retrieval for Open-Domain Question Answering.” arXiv:2004.04906.

3. Radford, A., Kim, J. W., et al. (2021). “Learning Transferable Visual Models from Natural Language Supervision (CLIP).” arXiv:2103.00020.

4. Li, J., et al. (2022). “BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation.” arXiv:2201.12086.

5. Reimers, N., & Gurevych, I. (2019). “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.” arXiv:1908.10084.

6. Johnson, J., Douze, M., & Jégou, H. (2019). “Billion-scale similarity search with GPUs.” IEEE Transactions on Big Data. arXiv:1702.08734.

7. Mialon, G., et al. (2023). “Augmenting LLMs with External Knowledge: A Survey of Retrieval-Augmented Generation.” arXiv:2306.13931.

8. Vaswani, A., et al. (2017). “Attention Is All You Need.” Advances in Neural Information Processing Systems, 30. arXiv:1706.03762.

9. LangChain Documentation. Available at: https://python.langchain.com/

10. Pinecone Documentation — Vector Database for Machine Learning. Available at: https://docs.pinecone.io/

11. Hugging Face. Transformers Documentation. Available at: https://huggingface.co/docs/transformers/

12. Hugging Face. Inference API Documentation. Available at: https://huggingface.co/docs/api-inference/

13. Meta AI. Llama 3.1 Model Card. Available at: https://huggingface.co/meta-llama

14. FastAPI Documentation. Available at: https://fastapi.tiangolo.com/

15. Streamlit Documentation. Available at: https://docs.streamlit.io/